

✓ Object-Oriented Programming (OOP) in Python

Welcome to this comprehensive guide to OOP in Python

▼ Table of Contents

1. Introduction to OOP
2. Core Concepts with Code Examples
3. Inheritance
4. Polymorphism
5. Encapsulation
6. Abstraction
7. Composition and Aggregation
8. Object LifeCycle and Memory
9. Overview of some Key OOP Design Patterns
10. Summary of SOLID Principles in OOP

1. Introduction to OOP

1.1 What is OOP?

► OOP is a programming paradigm (coding style) that organizes code around objects and classes rather than functions and logic. It allows you to model real-world entities as objects with properties (attributes) and behaviors (methods).

1.2 Why Use OOP?

- 1- Code Reusability:** Write once, use many times
- 2- Modularity:** Break complex problems into smaller, manageable pieces
- 3- Maintainability:** Easier to update and modify code
- 4- Scalability:** Better organization for large applications
- 5- Real-world Modeling:** Natural way to represent real-world concepts

✓ 2. Core Concepts with Code Examples

2.1 Classes and Objects

A) Class:

► Is The Template For Creating Objects [Object Constructor | Blueprint]

(*note*: Class Name Written With PascalCase [UpperCamelCase] Style)

B) Object:

► is an instance of a class and Have Their Methods and Attributes

(*Note: Everything in Python is Object*)

Example:

- **Class:** (Person)
 - A general blueprint for creating people objects.
 - **Object (Instance):** (Ahmed, Mona)
 - Specific individuals created from the Person class.
 - **Attributes (Data):** name - age - address - phone_number
 - **Methods (Behaviors / Functions):** walk() - eat() - sing()
-

```
# Class definition
class Person:
    print("A person has been added")

# Create object from person class
Ahmed = Person()

# Type of this object
print(type(Ahmed))
# check if the Ahmed is an object of Person Class or not
print(isinstance(Ahmed, Person))
```

```
➞ A person has been added
<class '__main__.Person'>
True
```

2.2 The `__init__` Method

- Is Called Every Time You Create Object From Class
- Initialize The Data (attributes) For The Object

(*Note*: Any Method like (`__ Method __`) Called Dunder or Magic Method)

```
class Member:
```

```
def __init__(self):  
  
    print("A New Member Has Been Added")  
  
member_one = Member()  
  
print(member_one.__class__)
```

➞ A New Member Has Been Added
<class '__main__.Member'>

2.3 The *self* Keyword

► refers to the current instance of the class. It's used to access instance variables and methods.

```
class Person:  
    def __init__(self, my_name, my_age):  
        self.name = my_name # self.name refers to this instance's name  
        self.age = my_age   # self.age refers to this instance's age  
  
    def introduce(self):  
        return f"Hi, I'm {self.name} and I'm {self.age} years old!"  
  
Abdullah = Person("Abdullah", 22)  
print(Abdullah.name)  
print(Abdullah.age)  
print(Abdullah.introduce())
```

➞ Abdullah
22
Hi, I'm Abdullah and I'm 22 years old!

2.4 The *Instance Attributes and Methods*

- **Instance Attributes:** Instance Attributes Defined Inside The Constructor
 - **Instance Methods:** Take Self Parameter Which Point To Instance Created From Class
 - Instance Methods can:-
 - Can Freely Access Attributes And Methods On The Same Object
 - Can Access The Class Itself
-

```

class Member:
    def __init__(self, first_name, middle_name, last_name, gender):
        # instance attributes
        self.fname = first_name
        self.mname = middle_name
        self.lname = last_name
        self.gender = gender

    # instance methods
    def full_name(self):
        return f"{self.fname} {self.mname} {self.lname}"

    def name_with_title(self):
        if self.gender == "Male":
            return f"Hello Mr {self.fname}"
        elif self.gender == "Female":
            return f"Hello Miss {self.fname}"
        else:
            return f"Hello {self.fname}"

    def get_all_info(self):
        return f"{self.name_with_title()}, Your Full Name Is: {self.full_name()}"

member_one = Member("Osama", "Mohamed", "Elsayed", "Male")
member_two = Member("Ahmed", "Ali", "Mahmoud", "Male")
member_three = Member("Mona", "Ali", "Mahmoud", "Female")

print(member_one.fname, member_one.mname, member_one.lname)
print(member_two.fname)
print(member_three.fname)

print(member_two.full_name())
print(member_two.name_with_title())

print(member_one.get_all_info())

```

```

➡ Osama Mohamed Elsayed
Ahmed
Mona
Ahmed Ali Mahmoud
Hello Mr Ahmed
Hello Mr Osama, Your Full Name Is: Osama Mohamed Elsayed

```

2.5 Instance vs Class Variables

- **Class Attributes:** shared by all instances
 - Modifying class Attributes affects all instances

- **Instance Attributes:** unique to each instance
-

```
class Employee:
    # class Attributes
    Company_name = "TechCorp"

    def __init__(self, name, salary):
        self.name_of_employee = name
        self.salary_of_employee = salary

    def info(self):
        return f"{self.name_of_employee} Work at {Employee.Company_name}"

##### TEST #####
emp1 = Employee("Ahmed", 5000)
emp2 = Employee("Mustafa", 6000)

print(emp1.info())
print(emp2.info())

# After Changing a class Attribute (Company_name)

Employee.Company_name = "New_Tech_222"

print(emp1.info())
print(emp2.info())
```

```
➡ Ahmed Work at TechCorp
   Mustafa Work at TechCorp
   Ahmed Work at New_Tech_222
   Mustafa Work at New_Tech_222
```

2.6 Methods vs Functions

- Method is a function defined inside the class that operate on objects.
-

```
class Calculator:
    def __init__(self):
        self.history = []

    # Instance method
    def add(self, a, b):
        result = a + b
        self.history.append(f"{a} + {b} = {result}")
        return result
```

```
# Function (outside class)
def multiply(a, b):
    return a * b
```

2.7 Class Methods & Static Methods

A) Class Methods:

- Marked With **@classmethod** Decorator To Flag It As Class Method
- It Take **cls** Parameter To Point To The Class not The Instance (not self)
- It Doesn't Require Creation of a Class Instance
 - ▶ Used When You Want To Do Something With The Class Itself

B) Static Methods:

- It Takes No Parameters
 - Its Bound To The Class Not Instance
 - ▶ Used When Doing Something Doesn't Have Access To Object Or Class But Related To Class
-

```
class student:
    school_name = "AI Academy"

    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    # A) Class Method
    @classmethod
    def change(cls, new_name):
        cls.school_name = new_name

    # B) Static Method
    @staticmethod
    def say_hello():
        print("Hello From Static Method")

    @staticmethod
    def is_passing(grade):
        return grade >= 50

#-----
```

```
s1 = student("Ahmed", 90)

student.change("DeepLearning Institute")

print(f"New School Name: {student.school_name}")

print(student.say_hello())
```



```
New School Name: DeepLearning Institute
Hello From Static Method
None
```

2.8 Magic Methods

- dunder or magic methods (`__MethodName__`)
 - They allow you To define how objects of your classes behave when interacting with built-in operations, functions, or language constructs.

TYPES:

- Initialization and destruction: `__init__`, `__new__`, `__del__`
- Representation: `__str__`, `__repr__`
- Comparison: `__eq__`, `__lt__`, `__gt__`
- Arithmetic operations: `__add__`, `__sub__`, `__mul__`
- Container behavior: `__len__`, `__getitem__`, `__setitem__`, `__contains__`
- Context management: `__enter__`, `__exit__`
- Attribute access: `__getattr__`, `__setattr__`, `__delattr__`
- Callables: `__call__`

Some Definitions:

- `__init__` :Called Automatically When Instantiating Class
- `self` `class` :The class to which a class instance belongs
- `__str__` :Gives a Human-Readable Output of the Object
- `__repr__` :Developer-friendly string representation
- `__len__` :Returns the Length of the Container Called When We Use the Built-in `len()` Function on the Object

```
class skills:
    def __init__(self):
```

```

self.my_skills = ["Html", "Css", "Js"]

def __len__(self):
    return len(self.my_skills)

profile = skills()
print(profile)

print(len(profile))

profile.my_skills.append("PHP")
profile.my_skills.append("MySQL")

print(len(profile))

```

```

↔ <__main__.skills object at 0x7e834bde4710>
3
5

```

```

# First Case: Without using __str__ or __repr__
class Book:

```

```

    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

```

```

book = Book("1984", "George Orwell", 328)
print(book)

```

```

print('_' * 40)
print('\n')

```

```

# Second Case: With using __str__ or __repr__
class Book:

```

```

    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

```

```

    def __str__(self):
        # User-friendly string representation
        return f'"{self.title}" by {self.author}'

```

```

    def __repr__(self):
        # Developer-friendly string representation

```



```
return f"Book('{self.title}', '{self.author}', {self.pages})"
```

```
book = Book("1984", "George Orwell", 328)
```

```
print(str(book))    # '1984' by George Orwell
```

```
print(repr(book))   # Book('1984', 'George Orwell', 328)
```

```
print(book)         # Calls __str__ by default: '1984' by George Orwell
```

```
➦ <__main__.Book object at 0x782fe676cd50>
```

```
'1984' by George Orwell
```

```
Book('1984', 'George Orwell', 328)
```

```
'1984' by George Orwell
```

✓ 3.Inheritance

► It is a mechanism that allows a class (child) to acquire properties and behaviors (attributes and methods) from another class (parent).

- This promotes code reuse, logical hierarchy, and scalability in applications.
 - Types:
 - Single Inheritance
 - Multilevel Inheritance
 - Multiple Inheritance
 - Hierarchical Inheritance
-

3.1 Single Inheritance

class A:

class B(A):

```
class Animal:
    def __init__(self, name, species):
        self.animal_name = name
        self.animal_species = species

    def make_sound(self):
        return f"{self.animal_name} make a sound"

    def info(self):
        return f"{self.animal_name} is a {self.animal_species}"
```

```
# Dog inherits from the Animal
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "dog") ## Calling parent constructor
        self.breed = breed

    def make_sound(self): # override method
        return f"{self.animal_name} barks"

    def fetch(self):      # new method specific to the Dog class
        return f"{self.animal_name} fetch the ball"

dog = Dog("Buddy", "Golden Retriever")
print(dog.info())        # (inherited method)
print(dog.make_sound())  # (overridden method)
print(dog.fetch())       # (new method)
```

3.2 Multilevel Inheritance

```
class A:
```

```
class B(A):
```

```
class C(B):
```

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

class CarA(Vehicle):
    def __init__(self, brand, model, doors):
        super().__init__(brand, model)
        self.doors = doors

    def honk(self):
        return "Beep Beep!"

class CarB(CarA):
    def __init__(self, brand, model, doors, Battery_capacity):
        super().__init__(brand, model, doors)
        self.Battery_capacity = Battery_capacity

    def charge(self):
        return f"Charging {self.Battery_capacity} KWh Battery"
```

```
tesla = CarB("Tesla", "Model 3", 4, 75)

print(tesla.honk())    # (from CarA)
print(tesla.charge())  # (from CarB)
```

3.3 Multiple Inheritance

```
class A:

class B:

class C(A, B):
```

```
class Flyable:
    def fly(self):
        return "Flying through the air"

class Swimmable:
    def swim(self):
        return "Swimming in water"

class Duck(Animal, Flyable, Swimmable):
    def __init__(self, name):
        super().__init__(name, "Duck")

    def make_sound(self):
        return f"{self.animal_name} quacks"

duck = Duck("Donald")
print(duck.info())
print(duck.make_sound())
print(duck.fly())
print(duck.swim())
```

3.4 Hierarchical Inheritance

```
class A:

class B(A):

class C(A):
```

```
class A:
    def print_a(self):
```

```
        print('Hello from class A')

class B(A):
    def print_b(self):
        print('Hello from class B')

class C(A):
    def print_c(self):
        print('Hello from class C')


c = C()
c.print_a()
c.print_c()

b = B()
b.print_a()
b.print_b()
```

3.5 The super() Function

► provides access to methods in a parent class from a child class.

```
class Shape:
    def __init__(self, color):
        self.color = color

    def describe(self):
        return f"A {self.color} shape"

class Rectangle(Shape):
    # Call parent constructor
    def __init__(self, color, width, height):
        super().__init__(color)
        self.width = width
        self.height = height

    # Call parent method
    def describe(self):
        parent_description = super().describe()
        return f"{parent_description} with area {self.width * self.height}"

rect = Rectangle("red", 5, 3)
print(rect.describe())
```

3.6 Method Overriding

```
class Payment:
    def __init__(self, amount):
        self.amount = amount

    def process(self):
        return f"Processing payment of ${self.amount}"

class CreditCardPayment(Payment):
    def __init__(self, amount, card_number):
        super().__init__(amount)
        self.card_number = card_number

    def process(self): # Override parent method
        return f"Processing credit card payment of ${self.amount} with card {self.card_number}"

class PayPalPayment(Payment):
    def __init__(self, amount, email):
        super().__init__(amount)
        self.email = email

    def process(self): # Override parent method
        return f"Processing PayPal payment of ${self.amount} to {self.email}"

payments = [
    CreditCardPayment(100, "1234567890123456"),
    PayPalPayment(75, "user@example.com")
]

for payment in payments:
    print(payment.process())
```

✓ 4. Polymorphism

► Polymorphism is an object-oriented programming concept where a single interface can represent different types of behaviors depending on the object that is using it. It allows different classes to define methods with the same name but with behavior appropriate to that class.

- This promotes flexibility and interchangeability in code.

Example:

- len() method Apply the Polymorphism Principle, when we pass:
 - string -> return number of characters
 - list -> return number of elements
 - dict -> return number of Keys
 - We use: => isinstance(obj_name, class_name)
-

```
class Duck:
    def make_sound(self):
        return "Quack"

    def move(self):
        return "Swimming"

class Dog:
    def make_sound(self):
        return "Woof"

    def move(self):
        return "Running"

class Robot:
    def make_sound(self):
        return "Beep"

    def move(self):
        return "Rolling"

def animal_actions(animal):
    # Function that works with any object that has make_sound and move methods
    print(f"Sound: {animal.make_sound()}")
    print(f"Movement: {animal.move()}")

animals = [Duck(), Dog(), Robot()]

for animal in animals:
    animal_actions(animal)
    print("---"*10)
```



```
Sound: Quack
Movement: Swimming
-----
Sound: Woof
Movement: Running
-----
```

Sound: Beep
Movement: Rolling

Applying polymorphism with inheritance + Lists

```
class BaseCountry:
    def name(self):
        pass

    def capital(self):
        pass

    def language(self):
        pass

class Egypt(BaseCountry):

    def name(self):
        print('Country: Egypt')

    def capital(self):
        print('Capital: Cairo')

    def language(self):
        print('Language: Arabic')

class Australia(BaseCountry):

    def name(self):
        print('Country: Australia')

    def capital(self):
        print('Capital: Canberra')

    def language(self):
        print('Language: English')

egypt = Egypt()
australia = Australia()

countries = [egypt, australia]
```

```
for country in countries:
    if isinstance(country, BaseCountry):
        country.name()
        country.capital()
        country.language()
```

```
↔ Country: Egypt
    Capital: Cairo
    Language: Arabic
    Country: Australia
    Capital: Canberra
    Language: English
```

✓ 5.Encapsulation

► is the bundling of data and methods that work on that data within a single unit, and restricting access to some components.

5.1 Public, Protected, Private Attributes

A) Public:

- Every Attribute and method can be modified and run from everywhere (inside and outside the class)

B) Protected:

- can be accessed from its class and subclass
- syntax: `_name`

C) Private:

- can be accessed from within the class or object only
- can't be modified from outside the class
- syntax: `__name`
- Note:
 - `_name` , `__name` => convention names in python for user

```
# public Example:
class Member:
    def __init__(self, name):
        self.name = name #public
```

```
one = Member("Ahmed")
print(one.name)
```

```
one.name = "Sayed"
```



```
print(one.name)
```

➡ Ahmed
Sayed

```
# protected Example:
class Member:
    def __init__(self, name):
        self._name = name # protected
```

```
one = Member("Ahmed")
print(one._name)
```

```
one._name = "Sayed"
print(one._name)
```

➡ Ahmed
Sayed

```
# Private Example:
class Member:
    def __init__(self, name):
        self.__name = name # Private
```

```
one = Member("Ahmed")
print(one.__name)
```

➡

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-9-3448242456.py in <cell line: 0>()
      5
      6 one = Member("Ahmed")
----> 7 print(one.__name)

AttributeError: 'Member' object has no attribute '__name'
```

```
# we can access the private
# _ClassName__PrivateVariable
class Member:
    def __init__(self, name):
        self.__name = name # Private

one = Member("Ahmed")
print(one._Member__name)
```

5.2 *setters, getters*

► Two methods allow me to modify and access private properties

```
class Member:
    def __init__(self, name):
        self.__name = name # private

    def say_hello(self):
        return f"Hello {self.__name}"

    # getters
    def get_name(self):
        return self.__name

    # setter
    def set_name(self, new_name):
        self.__name = new_name

one = Member("Ahmed")
one.get_name()

one.set_name("Abdullah")
one.get_name()
```

5.3 *Property Decorator*

```
class Member:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def say_hello(self):
        return f"Hello {self.name}"

    @property
    def age_in_days(self):
        return self.age * 365

one = Member("Ahmed", 40)

print(one.name)
```

```
print(one.age)
print(one.say_hello())
print(one.age_in_days)
```

✓ 6. Abstraction

► Abstraction is an object-oriented programming principle that involves hiding the internal details of how something works and showing only the essential features needed for interaction.

- It helps manage complexity by exposing only what is necessary and keeping the implementation hidden from the user.

6.1 Abstract Base Classes

► Provide an infrastructure for Defining Custom Abstract Base Classes.

- it has one or more abstract method
- by adding @abstractmethod Decorator on the method

```
from abc import ABCMeta, abstractmethod

class Shape(metaclass = ABCMeta):
    def __init__(self, color):
        self.color = color

    # Abstract method - must be implemented by subclasses
    @abstractmethod
    def area(self):
        pass

    # Abstract method - must be implemented by subclasses
    @abstractmethod
    def perimeter(self):
        pass

    # Concrete method - can be used by subclasses
    def display_info(self):
        return f"A {self.color} shape with area {self.area()}"

class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color)
        self.radius = radius
```

```

def area(self): # must
    return 3.14159 * self.radius ** 2

def perimeter(self): # must
    return 2 * 3.14159 * self.radius

# some_shape = Shape("red") # Error!!! (Can't instantiate abstract class Shape)

circle = Circle("blue", 5)

print(circle.perimeter()) # Abstract method
print(circle.display_info()) # Concrete method

```

```

↔ 31.4159
   A blue shape with area 78.53975

```

```

class Rectangle(Shape):
    def __init__(self, color, width, height):
        super().__init__(color)
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

rectangle = Rectangle("green", 4, 6)

print(rectangle.height)
print(rectangle.area())
print(rectangle.display_info())

```

```

↔ 6
   24
   A green shape with area 24

```

✓ 7. Composition and Aggregation

7.1 Composition

► Composition is a strong form of association where one class owns another class.

- The child object is created inside the parent and cannot exist independently.
 - If the parent is destroyed, the child is destroyed as well.
 - **Key Points:**
 - Strong ownership.
 - Life cycle of the child depends on the parent.
 - Typically created using: `self.child = ChildClass()` inside the parent.
-

```
class Salary:
    def __init__(self, pay, bonus):
        self.pay = pay
        self.bonus = bonus

    def annual_salary(self):
        return (self.pay * 12) + self.bonus

class Employee:
    def __init__(self, name, age, pay, bonus):
        self.name = name
        self.age = age
        self.obj_salary = Salary(pay, bonus) # Composition

    def total_salary(self):
        return self.obj_salary.annual_salary()

emp = Employee('max', 25, 15000, 10000)
print(emp.total_salary())
```

➡ 190000

7.2 Aggregation

► Aggregation is a weaker form of association where one class uses another class, but does not own it.

- The child object is passed into the parent and can exist independently.
 - **Key Points:**
 - Weak ownership.
 - Life cycle of the child is independent of the parent.
 - Typically implemented by passing the child object via constructor or setter.
-

```
class Salary:
    def __init__(self, pay, bonus):
```

```

    self.pay = pay
    self.bonus = bonus

    def annual_salary(self):
        return (self.pay * 12) + self.bonus

class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.obj_salary = salary # Aggregation

    def total_salary(self):
        return self.obj_salary.annual_salary()

salary = Salary(15000, 10000)
emp = Employee('max', 25, salary)
print(emp.total_salary())

```

↔ 190000

7.3 Comparison Between Composition, Aggregation and Inheritance

A) Composition:

- Strong relationship.
- The child object is created and owned inside the parent object.
- *Example:* Car has a Engine.

B) Aggregation:

- Weak relationship.
- The child object can exist independently of the parent.
- *Example:* University has Student.

C) Inheritance:

- "Is-a" relationship.
- *Example:* Dog is an Animal.

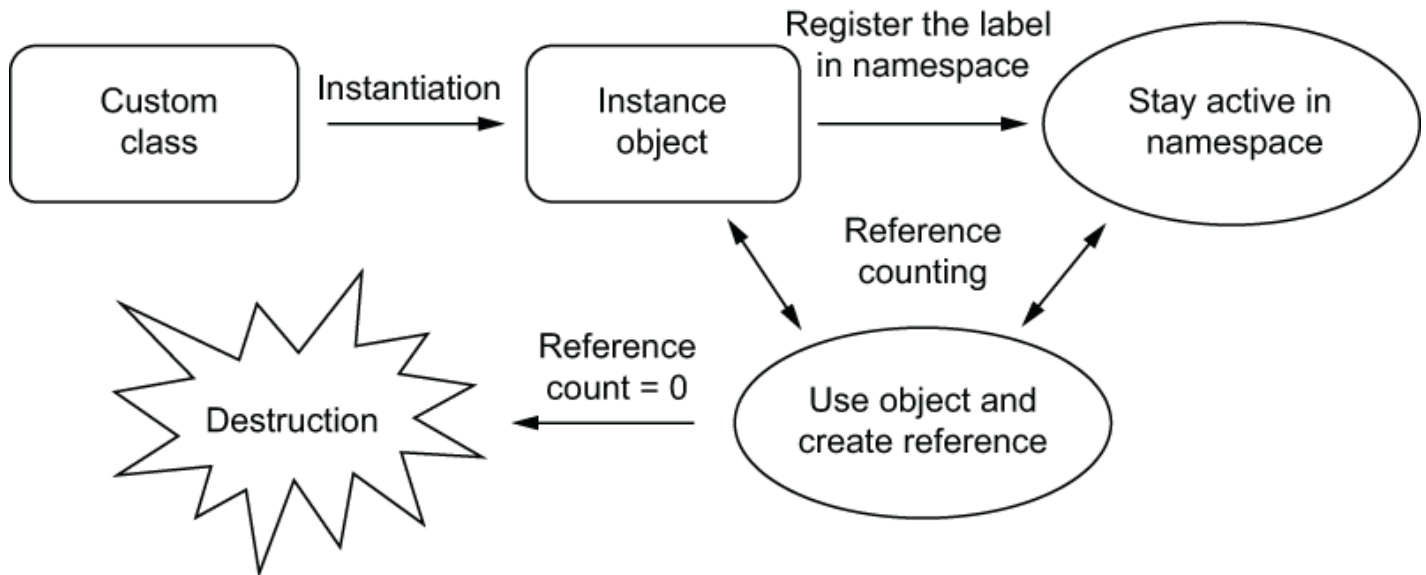
8. Object Lifecycle and Memory

In OOP, when we create an object from any class, this object goes through three main stages:

1- Creation: When it's first instantiated

2- Usage: While the object is being used in the program

3- Deletion: When it's no longer needed and removed from memory



- **Is Important For:**

- **Efficient Memory Management:** When you create objects, they occupy space in the computer's memory. If objects that are no longer in use aren't removed, they can accumulate and consume significant memory, leading to slow performance or even program crashes.
- **Performance Optimization:** Objects that use system resources (like open files, database connections) must be properly closed and released to ensure these resources aren't depleted.
- **Avoiding Errors:** A lack of understanding of memory management can lead to hard-to-trace errors, such as accessing an object that has already been deleted.

✓ 8.1 The `__del__` Method

► You can use it to clean up resources that the object might have used, such as closing open files or releasing database connections.

- However, you shouldn't rely on it completely for resource deallocation because Python doesn't guarantee exactly when it will be called.

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance
        print(f"Bank account number: {self.account_number} opened with a balance of : ")

    def __del__(self):
        # This method is called when the object is deleted
```

```
print(f"Closing bank account number: {self.account_number}...")
```

```
print(f"Bank account number: {self.account_number} closed.")
```

```
savings_account = BankAccount("123456789", 1000)
```

```
checking_account = BankAccount("987654321", 500)
```

```
# When there are no more references to savings_account, or when the program ends  
# __del__ will be called.
```

```
# We can expedite the deletion process by dereferencing the object:
```

```
del savings_account
```



```
Bank account number: 123456789 opened with a balance of $1000.  
Bank account number: 987654321 opened with a balance of $500.  
Closing bank account number: 123456789...  
Bank account number: 123456789 closed.
```

✓ 8.2 Garbage Collection in Python

► Is an automatic memory management system. Its job is to identify objects that are no longer reachable (there are no longer any references pointing to them) and free up the space they occupy in memory.

- This frees the programmer from the task of manual memory management, which can be complex and lead to errors.

NOTE:

- How does **Reference Counting** Work?.
 - When an object is created, its **reference count = 1**.
 - Each time a new reference is created that points to this object, the reference count increases (**reference count += 1**).
 - And each time a reference is deleted, the reference count decreases.(**reference count -= 1**)
 - When the reference count reaches zero (**reference count = 0**), the garbage collector knows that the object is no longer in use and can be deleted.

```
class Car:  
    def __init__(self):  
        print("Car created")
```

```
    def __del__(self):  
        print("Car deleted")
```

```
def test():
```



```
my_car = Car()
```

```
test() # After function ends, the object has no references → it's  
# __del__() will be called
```

⇒ Car created
Car deleted

✓ *9. Overview of some Key OOP Design Patterns *

► Design patterns are typical solutions to commonly occurring problems in software design. They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.

- You can't just find a pattern and copy it into your program, You can follow the pattern details and implement a solution that suits the realities of your own program.

9.1 Factory Pattern

► is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

```
class Dog:  
    def speak(self):  
        return "Woof!"  
  
class Cat:  
    def speak(self):  
        return "Meow!"  
  
class AnimalFactory:  
    def get_animal(self, animal_type):  
        if animal_type == "dog":  
            return Dog()  
        elif animal_type == "cat":  
            return Cat()  
  
factory = AnimalFactory()  
animal = factory.get_animal("dog")  
print(animal.speak())
```

⇒ Woof!

9.2 Singleton Pattern

► is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

```
class Singleton:
    # This class-level variable will hold the single instance
    _instance = None

    def __new__(cls):
        """
        __new__ is responsible for creating the object (before __init__ is called).
        We override it to ensure only one instance is ever created.
        """
        # Check if an instance already exists
        if cls._instance is None:
            print("Creating a new Singleton instance")
            # If not, create and store the instance
            cls._instance = super().__new__(cls)
        else:
            print("Reusing the existing Singleton instance")

        # Always return the single instance
        return cls._instance

    def __init__(self):
        print("Running __init__ (this may run multiple times)")

print("Creating the first object:")
a = Singleton() # This will create the object

print("\nCreating a second object:")
b = Singleton() # This will return the same object

# Compare both objects
print("\nAre 'a' and 'b' the same instance?")
print("Result:", a is b) # → True (both refer to the same object)

# Show memory addresses
print("ID of a:", id(a))
print("ID of b:", id(b))
```

➡ Creating the first object:
Creating a new Singleton instance

```
Running __init__ (this may run multiple times)
```

```
Creating a second object:
```

```
Reusing the existing Singleton instance
```

```
Running __init__ (this may run multiple times)
```

```
Are 'a' and 'b' the same instance?
```

```
Result: True
```

```
ID of a: 133932943328976
```

```
ID of b: 133932943328976
```

9.3 Strategy Pattern

► is a Behavioral Design Pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable.

- lets the algorithm vary independently from clients that use it
- Follows Open/Closed Principle
- **used When:**
 - When you want to switch between different algorithms at runtime.
 - Avoiding long **if/else** or **switch** statements.

```
class Add:
    def execute(self, a, b):
        return a + b

class Multiply:
    def execute(self, a, b):
        return a * b

class Calculator:
    def __init__(self, strategy):
        self.strategy = strategy

    def calculate(self, a, b):
        return self.strategy.execute(a, b)

calc = Calculator(Add())
print(calc.calculate(5, 3))

calc.strategy = Multiply()
print(calc.calculate(5, 3))
```

➡ 8
15

9.4 Observer Pattern

✓ 10. Summary of SOLID Principles in OOP

► SOLID is a set of five object-oriented design principles that can help you write more maintainable, flexible, and scalable code based on well-designed, cleanly structured classes

- RESOURCES: [link](#)

10.1 Single-responsibility principle (SRP)

A class should have only one reason to change.

- If a class takes care of more than one task, then you should separate those tasks into separate classes.
-

```
# A) ❌ Bad Version (Violates SRP)
# =====
# file_manager_srp.py

from pathlib import Path
from zipfile import ZipFile

class FileManager:
    def __init__(self, filename):
        self.path = Path(filename)

    def read(self, encoding="utf-8"):
        return self.path.read_text(encoding)

    def write(self, data, encoding="utf-8"):
        self.path.write_text(data, encoding)

    def compress(self):
        with ZipFile(self.path.with_suffix(".zip"), mode="w") as archive:
            archive.write(self.path)

    def decompress(self):
        with ZipFile(self.path.with_suffix(".zip"), mode="r") as archive:
            archive.extractall()
```

```
# ✅ Good Version (Follows SRP)
# =====
# file_manager_srp.py
```

```

from pathlib import Path
from zipfile import ZipFile

class FileManager:
    def __init__(self, filename):
        self.path = Path(filename)

    def read(self, encoding="utf-8"):
        return self.path.read_text(encoding)

    def write(self, data, encoding="utf-8"):
        self.path.write_text(data, encoding)

class ZipFileManager:
    def __init__(self, filename):
        self.path = Path(filename)

    def compress(self):
        with ZipFile(self.path.with_suffix(".zip"), mode="w") as archive:
            archive.write(self.path)

    def decompress(self):
        with ZipFile(self.path.with_suffix(".zip"), mode="r") as archive:
            archive.extractall()

```

10.2 Open-Closed Principle (OCP)

Software entities (classes, modules, functions, etc.) should be open for extension

- Open For -> Extension
- Closed For -> Modification

This means you should be able to add new functionality without altering existing, working code.

```

# A) ❌ Bad Version (Violates OCP)
# =====

from math import pi

class Shape:
    def __init__(self, shape_type, **kwargs):
        self.shape_type = shape_type
        if self.shape_type == "rectangle":

```

```
        self.width = kwargs["width"]
        self.height = kwargs["height"]
    elif self.shape_type == "circle":
        self.radius = kwargs["radius"]
```

```
def calculate_area(self):
    if self.shape_type == "rectangle":
        return self.width * self.height
    elif self.shape_type == "circle":
        return pi * self.radius**2
```

imagine that you need to add new shape like (square)
-> you have to modify the internal code and add another (elif)

B)  Good Version (Follows OCP)
=====

```
from abc import ABC, abstractmethod
from math import pi
```

```
class Shape(ABC):
    def __init__(self, shape_type):
        self.shape_type = shape_type

    @abstractmethod
    def calculate_area(self):
        pass
```

```
class Circle(Shape):
    def __init__(self, radius):
        super().__init__("circle")
        self.radius = radius
```

```
    def calculate_area(self):
        return pi * self.radius**2
```

```
class Rectangle(Shape):
    def __init__(self, width, height):
        super().__init__("rectangle")
        self.width = width
        self.height = height
```

```
    def calculate_area(self):
        return self.width * self.height
```

```
class Square(Shape):
```

```
def __init__(self, side):
    super().__init__("square")
    self.side = side

def calculate_area(self):
    return self.side**2
```

10.3 Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types.

- if a program works with a base class, it should work equally well when a derived class is substituted in its place, without knowing the substitution has happened.
-

```
# A) ❌ Bad Version (Violates LSP)
# =====

class Bird:
    def fly(self):
        print("Flying")

class Ostrich(Bird): # Ostrich can't fly!
    def fly(self):
        raise Exception("Can't fly!")

# Breaks LSP - can't substitute Ostrich for Bird
```

```
# B) ✅ Good Version (Follows LSP)
# =====

class Bird:
    pass

class FlyingBird(Bird):
    def fly(self):
        print("Flying")

class Ostrich(Bird): # No fly method
    def run(self):
        print("Running")
```

10.4 Interface Segregation Principle (ISP)

Clients should not be forced to depend upon methods that they do not use.
Interfaces belong to clients, not to hierarchies.

- this mean that rather than one large interface, it's better to have many small, specific interfaces.

```
# A) ❌ Bad Version (Violates ISP)
# =====

from abc import ABC, abstractmethod

class Printer(ABC):
    @abstractmethod
    def print(self, document):
        pass

    @abstractmethod
    def fax(self, document):
        pass

    @abstractmethod
    def scan(self, document):
        pass

class OldPrinter(Printer):
    def print(self, document):
        print(f"Printing {document} in black and white...")

    def fax(self, document):
        raise NotImplementedError("Fax functionality not supported in Old Printer")

    def scan(self, document):
        raise NotImplementedError("Scan functionality not supported in Old Printer")

class ModernPrinter(Printer):
    def print(self, document):
        print(f"Printing {document} in color...")

    def fax(self, document):
        print(f"Faxing {document}...")

    def scan(self, document):
        print(f"Scanning {document}...")

## it forces OldPrinter to expose an interface from Base Class that the class (OldPr:
```



```
# B)  Good Version (Follows ISP)
# =====

from abc import ABC, abstractmethod

class Printer(ABC):
    @abstractmethod
    def print(self, document):
        pass

class Fax(ABC):
    @abstractmethod
    def fax(self, document):
        pass

class Scanner(ABC):
    @abstractmethod
    def scan(self, document):
        pass

class OldPrinter(Printer):
    def print(self, document):
        print(f"Printing {document} in black and white...")

class NewPrinter(Printer, Fax, Scanner):
    def print(self, document):
        print(f"Printing {document} in color...")

    def fax(self, document):
        print(f"Faxing {document}...")

    def scan(self, document):
        print(f"Scanning {document}...")
```

10.5 Dependency Inversion Principle (DIP)

- 1- High-level modules should not depend on low-level modules. Both should depend on abstractions.
 - 2- Abstractions should not depend on details. Details should depend on abstractions.
-

```
# A)  Bad Version (Violates DIP)
# =====
```

```

class FrontEnd:
    def __init__(self, back_end): # tightly coupled to a specific class
        self.back_end = back_end

    def display_data(self):
        data = self.back_end.get_data_from_database()
        print("Display data:", data)

class BackEnd:
    def get_data_from_database(self):
        return "Data from the database"

# FrontEnd directly depends on BackEnd (a concrete class)
# => You cannot replace the data source (switch to API or CSV) without modifying FrontEnd

```

```

# B)  Good Version (Follows DIP)
# =====

from abc import ABC, abstractmethod

class FrontEnd:
    def __init__(self, data_source): # depends on abstraction
        self.data_source = data_source

    def display_data(self):
        data = self.data_source.get_data()
        print("Display data:", data)

# Abstract Interface
class DataSource(ABC):
    @abstractmethod
    def get_data(self):
        pass

# Concrete Implementations
class Database(DataSource):
    def get_data(self):
        return "Data from the database"

class API(DataSource):
    def get_data(self):
        return "Data from the API"

```

